

Primary Keys, Foreign Keys & JOINS

Foundations of Relational Databases

OMIS 105 · Introduction to DBMS

Agenda

1. **Relational databases** — tables, rows, columns
2. **Primary Key (PK)** — uniqueness guaranteed
3. **Foreign Key (FK)** — linking tables together
4. **Referential Integrity** — what FK enforces
5. **JOINS** — querying across tables
 - INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
6. **The IS NULL trick** — finding unmatched rows
7. **Quick Reference** — cheat sheet

Part 1

Relational Databases

What Is a Relational Database?

A **relational database** stores data in **tables** — rows and columns, like a spreadsheet — with formal rules that keep the data consistent.

departments

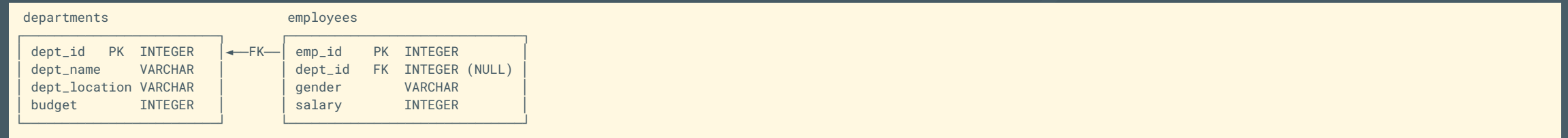
dept_id	dept_name	dept_location	budget
10	SALES	New York	500,000
20	ENGINEER.	San Jose	800,000
30	MARKETING	Chicago	400,000
40	LEGAL	Boston	300,000

Each **row** is one record. Each **column** is one attribute.

The power comes from linking tables together — that's where **keys** come in.

Our Example Schema

Two tables that model a simple company:



- `departments` is the **parent** table
- `employees` is the **child** table
- `employees.dept_id` is a **Foreign Key** pointing to `departments.dept_id`

Part 2

Primary Key (PK)

What Is a Primary Key?

A **Primary Key** is a column (or group of columns) that **uniquely identifies every row** in a table.

The three rules of a Primary Key

Rule		Meaning
Unique		No two rows may have the same PK value
Not NULL		A PK column can never be empty
Stable		PK values should not change once assigned

“💡 Think of a PK like a passport number — every person has exactly one, no two people share one, and it never changes.”

Primary Key — Example

```
CREATE TABLE departments (  
  dept_id      INTEGER PRIMARY KEY,  -- ← PK  
  dept_name    VARCHAR NOT NULL,  
  dept_location VARCHAR NOT NULL,  
  budget       INTEGER NOT NULL  
);
```

The data:

dept_id	dept_name	dept_location	budget
10	SALES	New York	500,000
20	ENGINEERING	San Jose	800,000
30	MARKETING	Chicago	400,000
40	LEGAL	Boston	300,000

dept_id is the PK — each value appears exactly once.

What Happens WITHOUT a Primary Key?

Without a PK, the database cannot prevent **duplicate rows**.

```
-- No PK defined - dangerous!  
INSERT INTO employees VALUES (101, 10, 'MALE', 95000);  
INSERT INTO employees VALUES (101, 10, 'MALE', 95000); -- accepted!
```

Result:

emp_id	dept_id	gender	salary
101	10	MALE	95,000
101	10	MALE	95,000

“
⚠ Now any salary total or JOIN on `emp_id` will give wrong answers.
Always define a Primary Key.
”

Primary Key Enforcement

With a PK defined, the database rejects duplicates automatically:

```
CREATE TABLE employees (  
    emp_id INTEGER PRIMARY KEY,    -- ← PK enforced  
    ...  
);  
  
INSERT INTO employees VALUES (101, 10, 'MALE', 95000);    -- OK  
INSERT INTO employees VALUES (101, 10, 'MALE', 95000);    -- ERROR!
```

```
Constraint Error: Duplicate key "emp_id: 101"  
violates primary key constraint.
```

✓ The database caught the problem — no bad data entered.

Part 3

Foreign Key (FK)

What Is a Foreign Key?

A **Foreign Key** is a column in one table that **references the Primary Key of another table**.

```
departments.dept_id (PK) ← employees.dept_id (FK)
```

The FK creates a **parent-child relationship**:

- `departments` is the **parent** — it owns the `dept_id` values
- `employees` is the **child** — it borrows `dept_id` to say which dept an employee belongs to

“💡 A FK is like a reference in a book — it points to something that must exist. If the thing it points to doesn't exist, the reference is broken.”

Declaring a Foreign Key

```
CREATE TABLE employees (  
  emp_id INTEGER PRIMARY KEY,  
  dept_id INTEGER REFERENCES departments(dept_id), -- ← FK  
  gender VARCHAR NOT NULL,  
  salary INTEGER NOT NULL  
);
```

The keyword `REFERENCES departments(dept_id)` tells DuckDB:

“Every non-NULL value in `employees.dept_id` must exist in `departments.dept_id`.”

Referential Integrity — What FK Prevents

Attempted action	Without FK	With FK
Insert employee with dept_id = 99 (doesn't exist)	Silently accepted ✗	Error raised ✓
Delete dept_id = 10 while employees reference it	Silently accepted ✗	Error raised ✓
Insert employee with dept_id = NULL	Accepted	Accepted ✓

Referential integrity means the database guarantees that every FK value either:

- points to a valid PK in the parent table, **or**
- is NULL

FK Enforcement in Action

```
-- ❌ Invalid: dept 99 does not exist  
INSERT INTO employees VALUES (999, 99, 'MALE', 50000);
```

Constraint Error: Violates foreign key constraint

```
-- ❌ Invalid: dept 10 still has children  
DELETE FROM departments WHERE dept_id = 10;
```

Constraint Error: Violates foreign key constraint

```
-- ✅ Valid: NULL means "not yet assigned"  
INSERT INTO employees VALUES (501, NULL, 'FEMALE', 72000);
```

1 row inserted.

NULL in a Foreign Key

A FK column **may be NULL** — it simply means the row has no parent yet.

In our data:

emp_id	dept_id	gender	salary
401	NULL	MALE	78,000
402	NULL	FEMALE	82,000

Employees 401 and 402 are **valid rows** — they just haven't been assigned to a department yet.

“ This is common in real systems: a new hire exists in the employee table before a department assignment is made. ”

Parent Table Must Exist First

Always create the **parent** before the **child**:

```
-- Step 1: parent table (no FK here)
CREATE TABLE departments ( dept_id INTEGER PRIMARY KEY, ... );

-- Step 2: child table (references parent)
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    dept_id INTEGER REFERENCES departments(dept_id),
    ...
);
```

And drop in **reverse** order:

```
DROP TABLE employees; -- child first
DROP TABLE departments; -- parent second
```

Part 4

JOINS

Why Do We Need JOINS?

Data is spread across multiple tables on purpose (**normalisation**).
A **JOIN** lets us combine columns from two or more tables for a query.

```
SELECT ...  
FROM left_table  
<JOIN TYPE> right_table  
ON left_table.key = right_table.key
```

In our schema the condition is always:

```
ON employees.dept_id = departments.dept_id
```

The three JOIN types we cover

Type	Returns
INNER JOIN	Only rows that match on both sides
LEFT JOIN	All left rows + matched right data
RIGHT JOIN	All right rows + matched left data

Visualising the Three JOINS

employees (left)			departments (right)		
emp_id	dept_id		dept_id	dept_name	
101	10	↔	10	SALES	← INNER match
102	10	↔	10	SALES	← INNER match
201	20	↔	20	ENGINEERING	← INNER match
202	20	↔	20	ENGINEERING	← INNER match
301	30	↔	30	MARKETING	← INNER match
302	30	↔	30	MARKETING	← INNER match
401	NULL	- x			← LEFT only
402	NULL	- x			← LEFT only
		x -	40	LEGAL	← RIGHT only

- **INNER JOIN** → 6 rows
- **LEFT JOIN** → 8 rows (all employees)
- **RIGHT JOIN** → 7 rows (all departments)

INNER JOIN

Returns only rows where the join condition matches on both sides.

```
SELECT e.emp_id,  
       d.dept_name,  
       d.dept_location,  
       e.gender,  
       e.salary  
FROM employees AS e  
INNER JOIN departments AS d  
  ON e.dept_id = d.dept_id  
ORDER BY e.emp_id;
```

emp_id	dept_name	dept_location	gender	salary
101	SALES	New York	MALE	95,000
102	SALES	New York	FEMALE	105,000
201	ENGINEERING	San Jose	MALE	130,000
...	...	...	...	...

“🔍 Employees 401/402 (NULL dept) and dept 40 (LEGAL) are both **excluded**.”

LEFT JOIN

Returns all rows from the LEFT table.
Matched rows get right-side columns filled in; unmatched rows get NULL.

```
SELECT e.emp_id, d.dept_name, e.gender, e.salary
FROM employees AS e
LEFT JOIN departments AS d
ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

emp_id	dept_name	gender	salary
101	SALES	MALE	95,000
...	...	...	...
401	NULL	MALE	78,000
402	NULL	FEMALE	82,000

“🔍 All 8 employees appear. Dept 40 (LEGAL) is still excluded.”

RIGHT JOIN

Returns all rows from the RIGHT table.
Matched rows get left-side columns filled in; unmatched rows get NULL.

```
SELECT d.dept_id, d.dept_name, e.emp_id, e.salary
FROM employees AS e
RIGHT JOIN departments AS d
ON e.dept_id = d.dept_id
ORDER BY d.dept_id;
```

dept_id	dept_name	emp_id	salary
10	SALES	101	95,000
...	...	...	...
40	LEGAL	NULL	NULL

“🔍 All 4 departments appear. Employees 401/402 (NULL dept) are excluded.”

The IS NULL Trick

Combine a LEFT or RIGHT JOIN with `WHERE ... IS NULL` to find **unmatched rows only**.

Find employees with no department

```
SELECT e.emp_id, e.salary
FROM employees AS e
LEFT JOIN departments AS d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;      -- ← only the unmatched left rows
```

Find departments with no employees

```
SELECT d.dept_id, d.dept_name
FROM employees AS e
RIGHT JOIN departments AS d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;      -- ← only the unmatched right rows
```

“ These patterns are extremely useful for data quality checks.

”

JOIN Comparison — At a Glance

	INNER JOIN	LEFT JOIN	RIGHT JOIN
Unassigned employees (401, 402)	✗ excluded	✓ included	✗ excluded
Empty department (LEGAL)	✗ excluded	✗ excluded	✓ included
Rows returned (our data)	6	8	7
Use when...	Need full match	Keep all left rows	Keep all right rows

“ **Tip:** A RIGHT JOIN is equivalent to swapping the table order and writing a LEFT JOIN. Most developers prefer LEFT JOINS for readability. ”

Part 5

Quick Reference

Primary Key — Cheat Sheet

```
-- Define a PK at column level
CREATE TABLE t (
  id INTEGER PRIMARY KEY,
  name VARCHAR
);

-- Define a composite PK at table level
CREATE TABLE order_items (
  order_id INTEGER,
  product_id INTEGER,
  quantity INTEGER,
  PRIMARY KEY (order_id, product_id)
);
```

Rules to remember:

- One PK per table
- Values must be unique and not NULL
- Use it as the target of Foreign Keys in child tables

Foreign Key — Cheat Sheet

```
-- Inline syntax
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    dept_id INTEGER REFERENCES departments(dept_id)
);

-- Table-level syntax (more explicit)
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    dept_id INTEGER,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Rules to remember:

- FK value must exist in the parent PK column, or be NULL
- Create parent table before child table
- Drop child table before parent table

JOIN — Cheat Sheet

```
-- INNER JOIN (matched rows only)
SELECT * FROM employees AS e
  INNER JOIN departments AS d ON e.dept_id = d.dept_id;

-- LEFT JOIN (all employees, dept info where available)
SELECT * FROM employees AS e
  LEFT JOIN departments AS d ON e.dept_id = d.dept_id;

-- RIGHT JOIN (all departments, employee info where available)
SELECT * FROM employees AS e
  RIGHT JOIN departments AS d ON e.dept_id = d.dept_id;

-- Find unmatched rows (LEFT + IS NULL)
SELECT * FROM employees AS e
  LEFT JOIN departments AS d ON e.dept_id = d.dept_id
 WHERE d.dept_id IS NULL;
```

Key Concepts — Summary

Concept	One-line definition
Primary Key	Uniquely identifies every row; never NULL
Foreign Key	References a PK in another table; may be NULL
Referential Integrity	FK values must point to existing PK values (or be NULL)
INNER JOIN	Rows matching on both sides only
LEFT JOIN	All left rows; NULLs for unmatched right columns
RIGHT JOIN	All right rows; NULLs for unmatched left columns
IS NULL trick	Filter a LEFT/RIGHT JOIN to find unmatched rows

“ **Golden rule:** Every table should have a Primary Key.
Every relationship between tables should be enforced by a Foreign Key.

”

That's a Wrap!

Practice notebooks

`PRIMARY_KEY/primary_key.ipynb` — PK deep dive

`FK_JOINS/fk_joins.ipynb` — FK + INNER / LEFT / RIGHT JOIN

Further reading

- [DuckDB SQL docs](#)
- [SQLZoo — interactive SQL exercises](#)
- [Mode SQL Tutorial](#)